



# WAVE3 COLLECTIVE

E-COMMERCE, ORDER & INVENTORY PLATFORM

Case Study — Design, Build & Deployment

[wave3collectiveph.com](https://wave3collectiveph.com)

Full-stack build — 2026

# Project Overview

Wave3 Collective is a Philippine streetwear brand selling limited-run embroidered tees. The founder needed a way to take orders online, track stock accurately, collect payment proof, and keep the books straight — without the cost of an off-the-shelf e-commerce platform. I designed and built a custom order, inventory, and bookkeeping system from scratch, then deployed it to a free hosting stack and connected it to the brand's own domain.

|                                                 |                                                |                                                   |
|-------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| <b>100%</b><br>Custom-built, no e-commerce SaaS | <b>PHP 0</b><br>Hosting cost (free-tier stack) | <b>Sold out</b><br>First drop sold out in ~2 days |
|-------------------------------------------------|------------------------------------------------|---------------------------------------------------|

## My Role

Sole developer — product design, backend engineering, frontend build, database architecture, and production deployment. Worked directly with the founder through iterative rounds of feedback, shipping features live as the business scaled from zero to its first sold-out drop.

## Tech Stack

- Node.js + Express — REST API server
- Vanilla JavaScript — no frontend framework, kept the codebase lean and fast
- libSQL / Turso — serverless cloud SQLite database (async, edge-replicated)
- Render — free-tier web hosting with auto-deploy from GitHub
- GitHub — version control, CI trigger for deployments
- Custom domain — wave3collectiveph.com via GoDaddy DNS

## The Problem

Limited-run drops meant stock could sell out in minutes, but there was no system to prevent overselling, no reliable way to track which orders were actually paid, and no clean separation between income, shipping costs, and profit. Manual tracking through chat messages and spreadsheets didn't scale past a handful of orders.

# Key Features

## Shopee-style stock reservation

Stock is reserved the instant a customer places an order — not at payment — matching the buying behavior customers already expect from major marketplace apps. Unpaid orders auto-expire after a configurable payment window and silently restock, so joy-buyers never lock up inventory for long. A live countdown keeps customers honest about the deadline.

## Shipping-quote gating

When a shipping fee needs manual confirmation, the payment timer doesn't start and payment channels stay hidden until the seller sets the fee — so customers are never asked to pay an incomplete total. A flat PHP 125 default keeps most orders instantly payable.

## Payment proof to verified receipt

Customers upload a screenshot of their payment; the admin reviews and marks the order paid, which atomically logs the sale as income, deducts stock, and generates a shareable receipt page with courier tracking.

## Admin dashboard

A single control panel for orders (filterable by status), inventory (stock, sizing, product photos), and money — with manual/historical order entry for accurate retroactive bookkeeping.

## Automatic bookkeeping

Every paid order logs its sale as income and its exact shipping cost as an expense automatically — no manual entry, no drift between what shipped and what was recorded. The Money tab surfaces Merch Sales, Shipping Collected, Total Expenses, and Net Profit at a glance, plus date-range reports exportable to Excel/CSV.

## Multi-currency for customers

Customers can view prices in PHP, USD, or USDT with live exchange-rate conversion, while every internal record — receipts, income, reports — stays fixed in the store's base currency to keep the books consistent.

### Built for zero-cost hosting

Payment-proof screenshots, QR codes, and product photos are stored directly in the database (not on disk) so the app runs correctly on free-tier hosting, where local disk storage doesn't persist between deploys.

### Order tracking for customers

Every order gets a short, human-typeable code (e.g. W3-A7XQ2M) that customers can look up anytime to see live status, courier tracking number, and — if something goes wrong — a direct link to message the brand's support channels.

## Notable Engineering Decisions

- **No overselling under race conditions** — stock decrements happen inside a database transaction at the moment of reservation, tested by simulating two simultaneous buyers competing for the last unit.
- **Async-first data layer** — the entire backend was migrated from a synchronous local SQLite driver to an async libSQL client to support a cloud-hosted database, with zero changes to frontend code.
- **Idempotent backfills** — when the auto-expense feature shipped after the store had already processed real orders, a boot-time backfill safely filled in the missing records without touching orders already logged.
- **Defensive input validation** — every user-editable money field (shipping fee, prices) is validated on both client and server to prevent a stray non-numeric entry from silently becoming free shipping.

## Results

The platform went live and processed its first product drop end-to-end — orders, payment proof, stock deduction, and shipping — with no manual bookkeeping required. The founder's first run of embroidered tees sold out in roughly two days, prompting the brand to register its own domain, [wave3collectiveph.com](https://wave3collectiveph.com), for the storefront.

### What's Next

Planned next phase: a fully custom marketing homepage matching the brand's visual direction, additional storefront pages (Collection, About), and an expanded multi-item cart experience.

Live storefront: [wave3collectiveph.com](https://wave3collectiveph.com)